

Backend Proxy & Secret-Handling Architecture Writeup

All OpenAI calls in ReadRight 2.0 are routed through a thin backend proxy: a single-file Express server (backend/server.js, Node + OpenAI). The Flutter app never talks to OpenAI directly and never possesses the API key. The app sends an HTTP request to the proxy's /generate-story endpoint; the proxy attaches the key server-side, calls the model, and returns only the generated story text to the app. In M3 the endpoint will instead accept structured fields (reading level, dolch words, interest from a fixed topic list) and the server will assemble the full prompt. This makes the content-safety constraints unbyypassable from the client and shrinks the attack surface of the endpoint.

During development the whole system runs locally in two terminals: one runs the proxy (npm start, port 3000 by default) and the other runs the Flutter app, which points at the proxy's local address.

We chose a proxy because a Flutter binary cannot safely hold a secret: anything compiled into the app, including values injected at build time, can be extracted from the distributed binary. A proxy keeps the key on a machine we control, gives us one chokepoint to enforce limits and logging, and lets us change models, prompts, or providers without shipping a new app build. It also matches our content-safety design: the model is only reachable through an endpoint whose behavior we define, not from arbitrary client code. We chose a thin Express proxy run locally on each developer's machine over a deployed, always-on backend because our timeline is one week, our demo and grading environment is a locally run app, and a local proxy gives us the security property that matters. The cost of this approach is that the key is replicated onto each teammate's machine rather than living on a single server, the app only functions while a teammate has the backend terminal running, and per-IP rate limiting is less meaningful when every client is localhost. This will hold up until the app must run somewhere that cannot reach a developer's localhost (ie. a phone off the dev network, a hosted demo, or real classroom users).

The key exists only in one file: backend/.env on each member's local computer. It is read at startup via dotenv. The server fails fast if the key (or the mandated model name, OPENAI_MODEL) is missing: it logs an error and exits rather than running in a broken state. Keeping the model name in the environment alongside the key also enforces the mini-tier requirement in one place instead of scattering model strings through code.

The key is never committed to repo. The .env file is mentioned across both .gitignores (one is base level, one on a database level).